

5

Аппарат сетей Петри

5.1 Введение в сети Петри

5.1.1 История создания

Первую идею сетей Петри Карл Адам Петри сформулировал в возрасте 13 лет для описания химических реакций. Официальной датой рождения теории считается 1962 год, когда он защитил докторскую диссертацию «Kommunikation mit Automaten» («Взаимодействие с автоматами») в Боннском университете. За свою работу он получил премию Тьюринга (ACM Turing Award) — высшую награду в области информатики.

5.1.2 Общая информация

Сеть Петри есть математический аппарат для моделирования дискретных систем. Графически она представляется как двудольный ориентированный граф двух типов вершин: *позиции* (круги) и *переходы* (прямоугольники).

- Позиции (Places) суть пассивные элементы, описывающие состояние системы (например, наличие ресурса или выполнение условия).
- Внутри позиции могут находиться фишки (tokens) — неотрицательное целое число, указывающее на количество ресурсов.

- Переходы (Transitions) суть активные элементы, описывающие события или действия системы.
 - Они могут срабатывать, изменяя состояние модели.
- Дуги (Arcs) суть направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами), которые показывают, как состояние влияет на события и наоборот.
- Маркировка (Marking) есть распределение фишек по позициям в определённый момент времени, то есть текущее состояние модели.
 - Смена маркировок происходит при срабатывании переходов в соответствии с определёнными правилами.

Теория сетей Петри, появившаяся как инструмент для анализа химических процессов, сегодня является мощным и наглядным математическим аппаратом. Она незаменима везде, где нужно описать параллельные, асинхронные и распределённые системы. В её основе лежат всего четыре элемента, а богатство поведения возникает из их комбинации.

5.1.3 Базовые элементы и принцип работы

В основе сети Петри лежит двудольный ориентированный мультиграф — это означает, что вершины в ней делятся на два типа, дуги всегда соединяют вершины разных типов, и между двумя вершинами может быть несколько дуг.

5.1.3.1 Позиции (Places) — «Условия»

Пассивные элементы, которые обозначаются кружками. Они описывают состояние системы, наличие ресурса или выполнение некоторого условия.

Метки (Tokens): Внутри позиции могут находиться фишки (маркеры), обозначаемые точками или числом. Они символизируют количество ресурсов или кратность выполнения условия. Состояние сети определяется распределением фишек по всем позициям и называется маркировкой.

5.1.3.2 Переходы (Transitions) — «События»

Активные элементы, обозначаемые прямоугольниками. Они моделируют действия, события или процессы, которые могут произойти в системе.

5.1.3.3 Дуги (Arcs) — «Связи»

Направленные связи между позициями и переходами.

- Входные дуги (от позиции к переходу) указывают, какие условия и сколько фишек необходимо для срабатывания события.
- Выходные дуги (от перехода к позиции) показывают, какие условия станут истинными после события и сколько фишек там появится.

5.1.3.4 Правило срабатывания перехода

Переход считается разрешённым (*enabled*), если каждая из его входных позиций содержит как минимум столько фишек, сколько указывает кратность входной дуги. Разрешённый переход может сработать (*fire*). Этот процесс:

- Удаляет из каждой входной позиции количество фишек, равное кратности входной дуги.
- Добавляет в каждую выходную позицию количество фишек, равное кратности выходной дуги.

Срабатывание перехода есть атомарная операция, которая происходит мгновенно и меняет маркировку сети. Если разрешено несколько переходов, любой из них может сработать.

5.1.4 Формальное определение

Сеть Петри (N-схема) — это четвёрка:

$$N = (P, T, F, M_0),$$

где:

- $P = p_1, p_2, \dots, p_n$ — конечное множество позиций (мест);
- $T = t_1, t_2, \dots, t_m$ — конечное множество переходов;
- $F \subseteq (P \times T) \cup (T \times P)$ — множество дуг (отношения инцидентности);
- $M_0 : P \rightarrow \mathbb{N}$ — маркировка, задающая начальное распределение фишек.

5.1.5 Свойства

Сети Петри используются не просто для моделирования, но и для проверки корректности системы. Для этого исследуются её важнейшие свойства:

- Ограниченность (Boundedness). Количество фишек в любой позиции сети никогда не превысит некоторого заданного предела K . Если $K = 1$, сеть называется безопасной.
- Активность (Liveness). Для любого перехода всегда существует достижимая маркировка, в которой он может сработать. Это гарантирует, что в системе не будет вечных блокировок.
- Достижимость (Reachability). Можно ли из начальной маркировки M_0 попасть в некоторую желаемую маркировку M_k .
- Сохраняемость (Conservation). Взвешенная сумма фишек по всем позициям остаётся постоянной (аналог закона сохранения массы или энергии).

5.1.6 Методы анализа сетей Петри

Для изучения этих свойств разработаны специальные методы:

- Граф (дерево) достижимости. Это фундаментальный метод. Строится ориентированный граф, вершинами которого являются все возможные маркировки, достижимые из начальной. Он наглядно показывает все возможные сценарии поведения системы.
- Матричные уравнения. Поведение сети можно описать с помощью матриц инцидентности. Срабатывание перехода соответствует прибавлению вектора, что позволяет решать задачи достижимости алгебраическими методами.
- Методы редукции. Сложная сеть упрощается путём замены стандартных подсетей (например, последовательных или параллельных фрагментов) на эквивалентные.

5.1.7 Расширения и модификации

Базовый аппарат очень абстрактен. Для решения прикладных задач были разработаны его расширения:

- Временные сети Петри (Timed Petri Nets). Каждому переходу приписывается время задержки. Это позволяет моделировать системы реального времени.

- Стохастические сети Петри (Stochastic Petri Nets). Времена задержек задаются распределениями случайных величин. Это мощный инструмент для анализа производительности систем, позволяющий вычислять показатели надёжности, пропускной способности и т.д.
- Раскрашенные сети Петри (Colored Petri Nets, CPN). Фишкам приписываются «цвета» (типы данных), а переходы могут иметь «выражения». Это позволяет моделировать большие системы с повторяющимися структурами в компактной форме.
- Иерархические сети Петри. Позволяют вкладывать одну сеть в переход или позицию другой, что облегчает моделирование сложных систем.
- Нечёткие сети Петри. Используются для систем с неопределённостью, где информация о событиях может быть неполной или неточной.

5.1.8 Где применяются

- Промышленность и производство. Моделирование гибких производственных линий, роботизированных ячеек, логистических цепочек.
- Информационные технологии. Моделирование распределённых баз данных, протоколов связи, параллельных алгоритмов и многопоточных программ.
- Бизнес-процессы. Описание и анализ workflow, документооборота, согласования заявок.
- Биология и медицина. Моделирование биохимических реакций, распространения эпидемий, работы медицинских систем.

5.1.9 Сравнение с другими моделями

5.1.9.1 Сети Петри vs. Конечные автоматы

Сети Петри естественным образом описывают параллелизм и синхронизацию, в то время как конечный автомат — это модель последовательного поведения. Для последовательных систем автомат может быть проще.

5.1.9.2 Сети Петри vs. Агентное моделирование

Сети Петри предлагают строгий математический аппарат для анализа (ограниченность, живость, достижимость) и сосредоточены на контроле и потоке управления. АВМ фокусируется на поведении индивидуальных сущностей и эмерджентных

свойствах. Эти подходы часто используются совместно, например, для моделирования сложных систем с помощью агентов, а их координации — с помощью сетей Петри.

5.2 Обедающие философы

Задача обедающие философы (Dining Philosophers)

5.2.1 История

Задача «Обедающие философы» была сформулирована Эдсгером Дейкстрой в 1965 году как иллюстрация проблемы синхронизации в параллельных вычислительных системах. Она наглядно демонстрирует явления взаимной блокировки (deadlock) и голодания (starvation) при конкурентном доступе к разделяемым ресурсам. Это одна из самых известных задач, демонстрирующая проблемы параллелизма, в частности, взаимные блокировки (deadlocks) и состояние гонки (race conditions).

5.2.2 Классическая постановка

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов (рис. 5.1).

Правила поведения философов:

- Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест.
- Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа.
- Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся.
- Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

Ограничения:

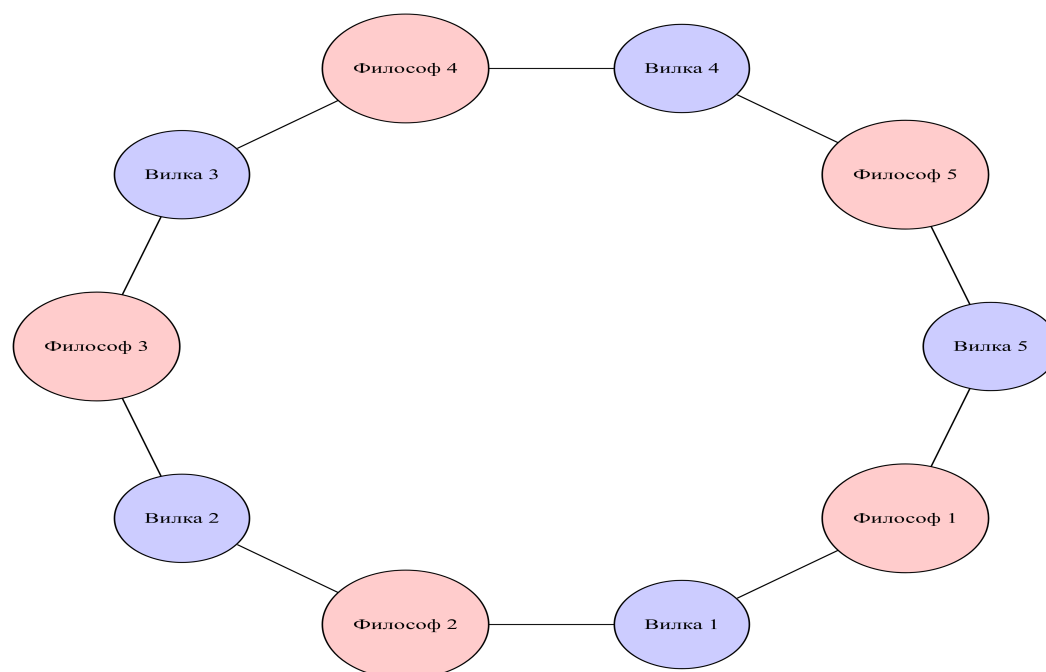


Рисунок 5.1. Схема Обедаящие философы

- Вилками нельзя пользоваться одновременно двум философам.
- Философ не может отнять вилку у соседа — только дожидаться, пока тот её положит.

5.2.3 Проблема взаимной блокировки (deadlock)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поесть — система замирает. Это и есть взаимная блокировка.

5.2.4 Решения проблемы

Для предотвращения deadlock предложены различные подходы:

- Арбитр (официант) — вводится дополнительный процесс, который разрешает есть не более чем $N-1$ философам одновременно.

- Асимметричное правило — философы с нечётным номером сначала берут левую вилку, затем правую, а чётные — наоборот.
- Атомарный захват — философ берёт обе вилки одновременно (если они свободны) иначе не берёт ни одной.
- Использование мьютексов и условных переменных в программных реализациях.

5.2.5 Связь с сетями Петри

Задача «Обедающие философы» является классическим примером для моделирования с помощью сетей Петри. Каждый философ и каждая вилка представляются позициями, а переходы соответствуют действиям: «взять левую вилку», «взять правую вилку», «начать есть», «положить вилки». Сеть Петри наглядно показывает возникновение deadlock и позволяет экспериментировать с различными механизмами синхронизации, например, введением арбитра.

5.2.6 Применение в реальных системах

Хотя задача абстрактна, её принципы применимы к любой системе, где несколько процессов конкурируют за несколько ресурсов, требующихся одновременно: доступ к базе данных, управление печатью на принтерах, распределение памяти, планирование задач в операционных системах. Понимание deadlock и способов его предотвращения есть фундаментальная часть параллельного программирования и проектирования вычислительных систем.

5.2.7 Цель лабораторной работы

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.

5.2.8 Общие замечания

- Стохастичность: алгоритм Гиллеспи вносит случайность. При каждом запуске могут получаться разные траектории и разное время наступления deadlock. Для надёжности рекомендуется проводить несколько прогонов (например, с разными seed) и усреднять результаты.
- Параметры N и tmax: для N=5 deadlock в классической модели обычно наступает в течение 50 единиц времени.
 - Если deadlock не обнаружился, можно увеличить tmax.
- Сеть с арбитром.
 - Теоретически она никогда не заходит в deadlock.
- Все три скрипта вместе дают полное исследование: от численного моделирования и визуализации динамики до сравнительного анализа и наглядной анимации.

5.2.9 Код модели

- src/DiningPhilosophers.jl

```

module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

# Определение простой структуры PetriNet

struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

```

```

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],
)
    incidence = zeros(Int, n_places, n_transitions)
    if isempty(place_names)
        place_names = [Symbol("p$i") for i = 1:n_places]
    end
    if isempty(transition_names)
        transition_names = [Symbol("t$i") for i = 1:n_transitions]
    end
    PetriNet(n_places, n_transitions, incidence, place_names,
        ↪ transition_names)
end

function add_arc!(net::PetriNet, place::Int, transition::Int,
    ↪ sign::Int)
    net.incidence[place, transition] += sign
end

# Построение сетей Петри

function build_classical_network(N::Int)
    n_places = 4N
    n_transitions = 3N
    net = PetriNet(n_places, n_transitions)

    for i = 1:N
        net.place_names[i] = Symbol("Think_$i")
        net.place_names[N+i] = Symbol("Hungry_$i")
        net.place_names[2N+i] = Symbol("Eat_$i")
        net.place_names[3N+i] = Symbol("Fork_$i")
    end
    for i = 1:N
        net.transition_names[i] = Symbol("GetLeft_$i")
        net.transition_names[N+i] = Symbol("GetRight_$i")
        net.transition_names[2N+i] = Symbol("PutForks_$i")
    end

    for i = 1:N
        think = i
        hungry = N + i
    end
end

```

```

    eat = 2N + i
    left_fork = 3N + i
    right_fork = 3N + (i % N + 1)

    get_left = i
    get_right = N + i
    put_forks = 2N + i

    add_arc!(net, think, get_left, -1)
    add_arc!(net, left_fork, get_left, -1)
    add_arc!(net, hungry, get_left, +1)

    add_arc!(net, hungry, get_right, -1)
    add_arc!(net, right_fork, get_right, -1)
    add_arc!(net, eat, get_right, +1)

    add_arc!(net, eat, put_forks, -1)
    add_arc!(net, think, put_forks, +1)
    add_arc!(net, left_fork, put_forks, +1)
    add_arc!(net, right_fork, put_forks, +1)
end

u0 = zeros(Float64, n_places)
for i = 1:N
    u0[i] = 1.0           # Think_i
    u0[3N+i] = 1.0       # Fork_i
end
return net, u0, net.place_names
end

function build_arbiter_network(N::Int)
    n_places = 4N + 1
    n_transitions = 3N
    net = PetriNet(n_places, n_transitions)

    for i = 1:N
        net.place_names[i] = Symbol("Think_$i")
        net.place_names[N+i] = Symbol("Hungry_$i")
        net.place_names[2N+i] = Symbol("Eat_$i")
        net.place_names[3N+i] = Symbol("Fork_$i")
    end
    net.place_names[4N+1] = :Arbiter

    for i = 1:N

```

```

    net.transition_names[i] = Symbol("GetLeft_$i")
    net.transition_names[N+i] = Symbol("GetRight_$i")
    net.transition_names[2N+i] = Symbol("PutForks_$i")
end

arbiter_idx = 4N + 1

for i = 1:N
    think = i
    hungry = N + i
    eat = 2N + i
    left_fork = 3N + i
    right_fork = 3N + (i % N + 1)

    get_left = i
    get_right = N + i
    put_forks = 2N + i

    add_arc!(net, think, get_left, -1)
    add_arc!(net, left_fork, get_left, -1)
    add_arc!(net, arbiter_idx, get_left, -1)
    add_arc!(net, hungry, get_left, +1)

    add_arc!(net, hungry, get_right, -1)
    add_arc!(net, right_fork, get_right, -1)
    add_arc!(net, eat, get_right, +1)

    add_arc!(net, eat, put_forks, -1)
    add_arc!(net, think, put_forks, +1)
    add_arc!(net, left_fork, put_forks, +1)
    add_arc!(net, right_fork, put_forks, +1)
    add_arc!(net, arbiter_idx, put_forks, +1)
end

u0 = zeros(Float64, n_places)
for i = 1:N
    u0[i] = 1.0
    u0[3N+i] = 1.0
end
u0[arbiter_idx] = N - 1
return net, u0, net.place_names
end

# Детерминированное моделирование (ODE)

```

```

function vectorfield(net::PetriNet, rates = ones(net.n_transitions))
    function f!(du, u, params, t)
        a = zeros(net.n_transitions)
        for j = 1:net.n_transitions
            rate = rates[j]
            prod = rate
            for i = 1:net.n_places
                if net.incidence[i, j] < 0
                    prod *= u[i] ^ (-net.incidence[i, j])
                end
            end
            a[j] = prod
        end
        du .= net.incidence * a
    end
    return f!
end

function simulate_ode(net::PetriNet, u0::Vector{Float64},
    ↪ tmax::Float64; saveat = 0.1)
    f = vectorfield(net)
    prob = ODEProblem(f, u0, (0.0, tmax))
    sol = solve(prob, Tsit5(), saveat = saveat)
    df = DataFrame(time = sol.t)
    for i = 1:net.n_places
        df[!, String(net.place_names[i])] = sol[i, :]
    end
    return df
end

# Стохастическое моделирование (алгоритм Гиллеспи)

function simulate_stochastic(
    net::PetriNet,
    u0::Vector{Float64},
    tmax::Float64;
    rates = ones(net.n_transitions),
    rng = Random.GLOBAL_RNG,
)
    u = copy(u0)
    t = 0.0
    times = [t]
    states = [copy(u)]

```

```
while t < tmax
    a = zeros(net.n_transitions)
    for j = 1:net.n_transitions
        rate = rates[j]
        prod = rate
        for i = 1:net.n_places
            if net.incidence[i, j] < 0
                prod *= u[i] ^ (-net.incidence[i, j])
            end
        end
        a[j] = prod
    end
    a0 = sum(a)
    if a0 == 0
        break
    end
    dt = -log(rand(rng)) / a0
    r = rand(rng) * a0
    cumsum = 0.0
    chosen = 1
    for j = 1:net.n_transitions
        cumsum += a[j]
        if r <= cumsum
            chosen = j
            break
        end
    end
    for i = 1:net.n_places
        u[i] += net.incidence[i, chosen]
    end
    t += dt
    if t <= tmax
        push!(times, t)
        push!(states, copy(u))
    end
end
df = DataFrame(time = times)
for i = 1:net.n_places
    df[!, String(net.place_names[i])] = [s[i] for s in states]
end
return df
end
```

Обнаружение deadlock

```
function detect_deadlock(df::DataFrame, net::PetriNet; tol = 1e-6)
    u_last = [df[end, String(net.place_names[i])] for i =
        ↪ 1:net.n_places]
    for j = 1:net.n_transitions
        can_fire = true
        for i = 1:net.n_places
            if net.incidence[i, j] < 0 && u_last[i] <
                ↪ -net.incidence[i, j] - tol
                can_fire = false
                break
            end
        end
        if can_fire
            return false
        end
    end
    return true
end
```

Визуализация

```
function plot_marking_evolution(df::DataFrame, N::Int)
    plots = []
    for group in ["Think", "Hungry", "Eat", "Fork"]
        p = plot(xlabel = "Time", ylabel = group, title = "$group
            ↪ states")
        for i = 1:N
            col = "$(group)_$i"
            if col in names(df)
                plot!(df.time, df[:, col], label = "$(group)_$i")
            end
        end
        push!(plots, p)
    end
    return plot(plots..., layout = (4, 1), size = (800, 1000))
end

end # module
```

5.2.10 Базовые эксперименты

- Файл: `scripts/dining_philosophers.jl`

5.2.10.1 Зачем нужен

Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:

- классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
- модифицированная модель с арбитром, которая должна предотвращать deadlock.

5.2.10.2 Что делает

- Создаёт классическую сеть Петри для $N=5$ философов с помощью `build_classical_network`.
- Запускает стохастическую симуляцию (алгоритм Гиллеспи) на время $t_{\max} = 50.0$.
- Сохраняет полную историю маркировок в CSV-файл (`dining_classic.csv`).
- Анализирует последнее состояние на предмет deadlock с помощью `detect_deadlock` и выводит результат в консоль.
- Строит графики эволюции маркировки по всем позициям (Think, Hungry, Eat, Fork) и сохраняет их как `classic_simulation.png`.
- Повторяет шаги 1–5 для сети с арбитром (`build_arbiter_network`), сохраняя результаты в `dining_arbiter.csv` и `arbiter_simulation.png`.

5.2.10.3 Входные данные

- Параметры внутри скрипта: $N = 5$, $t_{\max} = 50.0$.
- Функции модуля `DiningPhilosophers.jl`.

5.2.10.4 Выходные данные

- CSV-файлы в папке `data/`: полные траектории состояний (время + количество фишек в каждой позиции).
- Графики в папке `plots/`: `classic_simulation.png` и `arbiter_simulation.png`.
 - Четыре панели (Think, Hungry, Eat, Fork) с динамикой для каждого философа.
- Консольный вывод: сообщение `Deadlock обнаружен: true/false` для каждой модели.

5.2.10.5 Интерпретация результатов

- Классическая сеть (рис. 5.2) должна приводить к deadlock (через некоторое время все философы оказываются в состоянии Hungry, ни один не может есть).
 - Если `detect_deadlock` возвращает `true`, это подтверждает теоретический вывод: классическая постановка задачи обедающих философов ведёт к взаимной блокировке.
- Сеть с арбитром (дополнительная позиция с N-1 фишками) (рис. 5.3) должна избегать deadlock: `detect_deadlock` возвращает `false`.
 - Это демонстрирует один из способов синхронизации для предотвращения тупиков.
- Графики эволюции маркировки позволяют визуально увидеть, как меняются состояния.
 - В классической сети рано или поздно все `Eat_i` становятся нулевыми, а `Hungry_i` – единичными (или наоборот).
 - В сети с арбитром наблюдается постоянное чередование приёма пищи.

```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
```

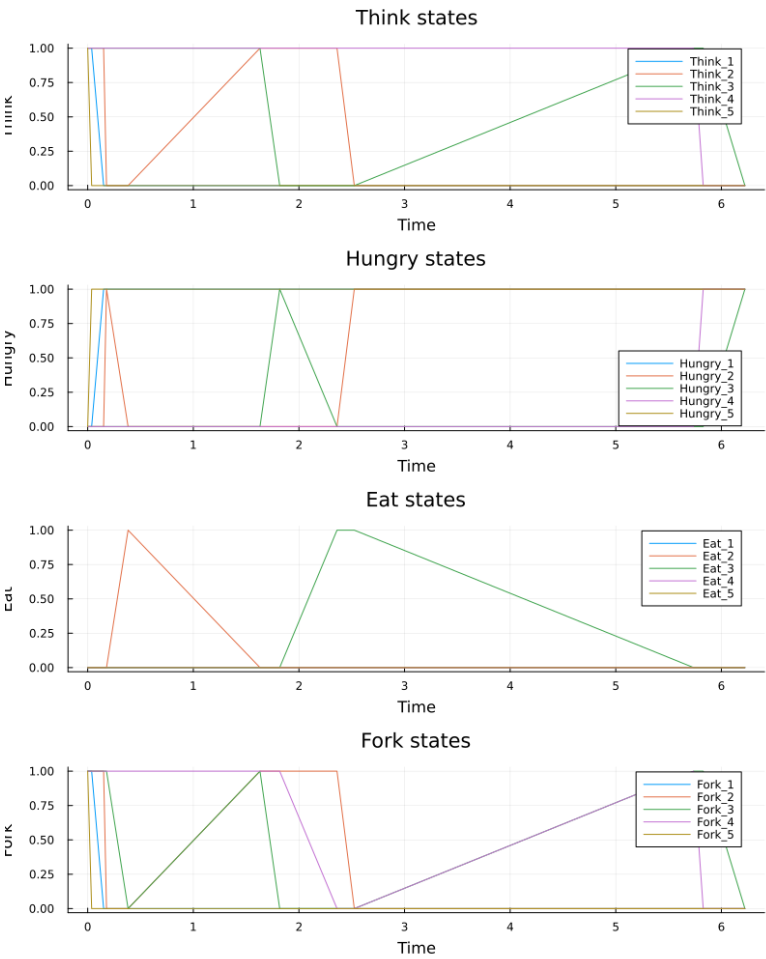
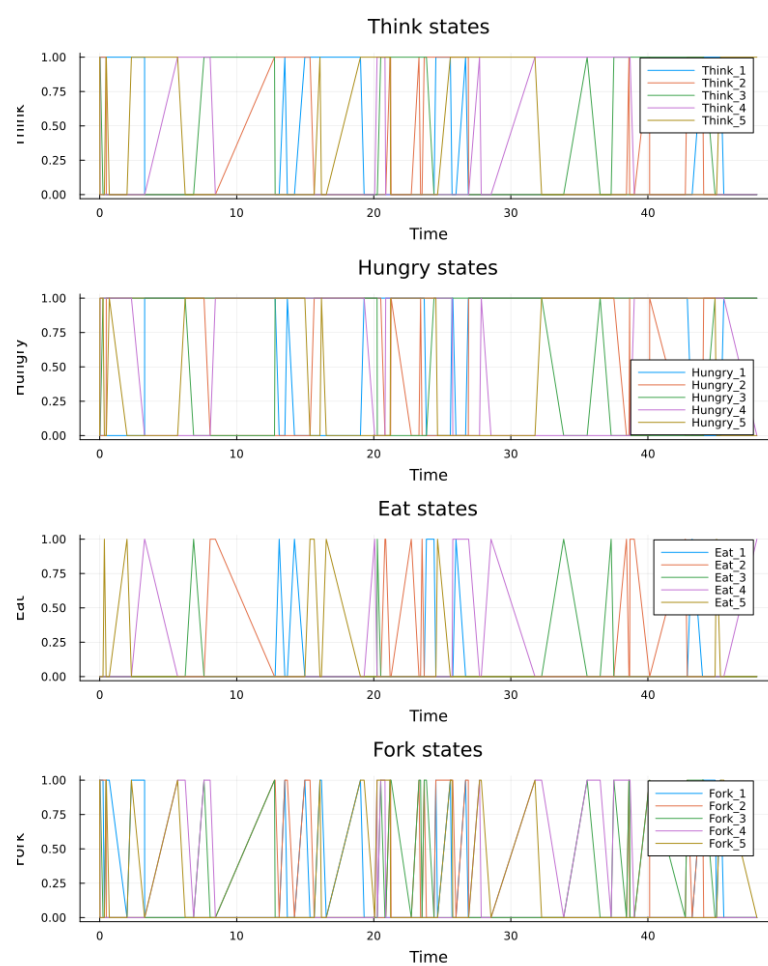


Рисунок 5.2. Классическая сеть

**Рисунок 5.3.** Сеть с арбитром

```
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

5.2.11 Анимация процесса

- Файл: `scripts/dining_philosophers_animation.jl`

5.2.11.1 Зачем нужен

- Для наглядной демонстрации динамики работы сети Петри во времени.
- Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели.

5.2.11.2 Что делает

- Берёт классическую сеть (для малого числа философов ($N=3$) для упрощения визуализации).
- Запускает стохастическую симуляцию на `tmax = 30.0`.
- Создаёт анимацию с помощью макроса `@animate`, где каждый кадр – это столбчатая диаграмма текущей маркировки (количество фишек в каждой позиции).
- Сохраняет анимацию как GIF-файл `philosophers_simulation.gif` с частотой 5 кадров в секунду.

5.2.11.3 Входные данные

- Параметры: $N = 3$, $t_{\max} = 30.0$, $\text{fps} = 5$.
- Результат симуляции из `simulate_stochastic`.

5.2.11.4 Выходные данные

- GIF-файл `plots/philosophers_simulation.gif`, показывающий изменение маркировки во времени (рис. 5.4).

5.2.11.5 Интерпретация результатов

- Анимация делает абстрактные состояния сети Петри живыми.
 - Видно, как фишки перемещаются между позициями: философы переходят из Think в Hungry, затем в Eat и обратно.
 - В какой-то момент в классической модели может наступить застывание – все переходы становятся неактивными (deadlock), что видно как неизменная маркировка на последних кадрах.
- Для сети с арбитром (если изменить скрипт, вызвав `build_arbiter_network`) анимация будет демонстрировать непрерывную работу без тупиков.
 - Это помогает понять механизм предотвращения deadlock.

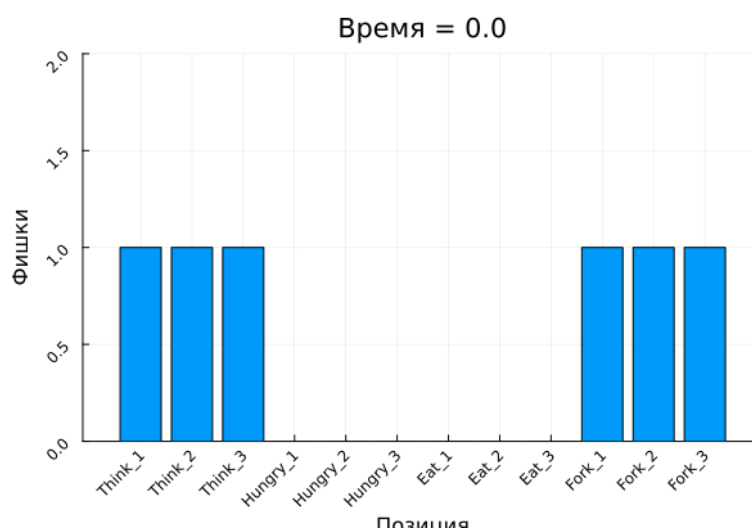


Рисунок 5.4. Динамика работы сети Петри

using DrWatson

```
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
end

gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")
```

5.2.12 Итоговый отчёт

— Файл: `scripts/dining_philosophers_report.jl`

5.2.12.1 Зачем нужен

- Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i).
- Скрипт генерирует сводный график.

5.2.12.2 Что делает

- Загружает ранее сохранённые CSV-файлы (`dining_classic.csv` и `dining_arbiter.csv`) из папки `data/`.
- Извлекает столбцы, соответствующие состоянию `Eat_i` для каждого философа.
- Строит два временных графика (один под другим): динамику «едящих» философов для классической сети и для сети с арбитром.
- Сохраняет объединённый график как `final_report.png` в папку `plots/`.

5.2.12.3 Входные данные

- CSV-файлы, созданные скриптом `dining_philosophers.jl`.
- Если их нет, скрипт выдаст ошибку.
- Поэтому эксперименты должны быть запущены до отчёта.

5.2.12.4 Выходные данные

- PNG-график `plots/final_report.png` с двумя панелями.

5.2.12.5 Интерпретация результатов

- Классическая сеть: на графике видно, что через некоторое время все линии `Eat_i` падают до нуля и остаются на нуле
 - Это и есть `deadlock`.
 - Философы больше не могут есть.
- Сеть с арбитром: линии `Eat_i` колеблются, всегда есть хотя бы один философ, который ест (или они поочерёдно получают доступ).
 - Отсутствие длительных нулевых периодов подтверждает, что система жива и не блокируется.
- График служит количественным доказательством эффективности арбитра (рис. 5.5).

```
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
```

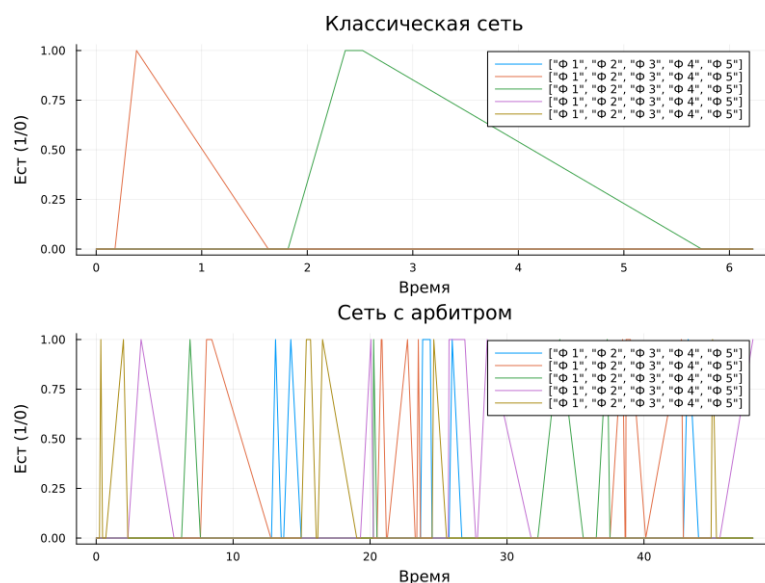


Рисунок 5.5. Итоговый отчёт

```
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5
```

```
# Столбцы для состояния "Ест"
eat_cols = [Symbol("Eat_<math>i</math>") for i = 1:N]
```

```
p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["Φ <math>i</math>" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)
p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["Φ <math>i</math>" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)
p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))
```



```
println("Отчёт сохранён в plots/final_report.png")
```

5.3 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

